

# 20 spark-submit

## What is `spark-submit` ?

`spark-submit` is the **official entry point** to run a Spark application on a cluster.

It:

- Starts a **new Spark driver**
- Requests resources from the **cluster manager**
- Launches **executors** on worker nodes
- Runs your Spark job end to end

👉 Every production Spark job is ultimately run using `spark-submit`.

---

## Why do we need `spark-submit` ?

In real projects:

- Jobs are **not run from notebooks**
- Jobs are executed via:
  - CI/CD pipelines
  - Airflow
  - Cron jobs
  - Shell scripts

`spark-submit` is what these systems call.

---

## Basic Syntax

```
spark-submit [options] <application> [app arguments]
```

Example:

```
spark-submit \  
--master spark://spark-master:7077 \  

```

```
--deploy-mode client \  
job.py
```

---

## Key Components Involved

When you run `spark-submit`:

1. A **new Driver JVM** is created
2. Driver connects to **Cluster Manager**
3. Cluster manager allocates **executors**
4. Executors register back with the driver
5. Driver schedules tasks
6. Job executes and exits

Each `spark-submit` = **one Spark application**

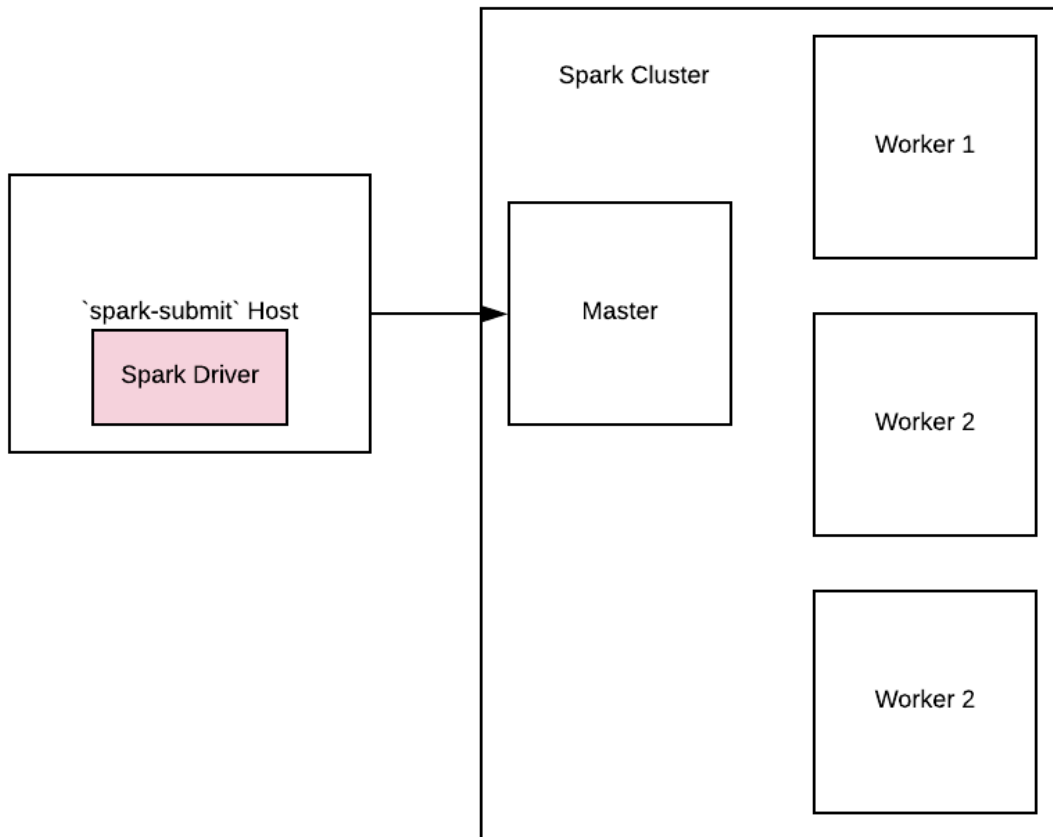
---

## Client vs Cluster Deploy Mode

### Client Mode

```
--deploy-mode client
```

--deploy-mode: client



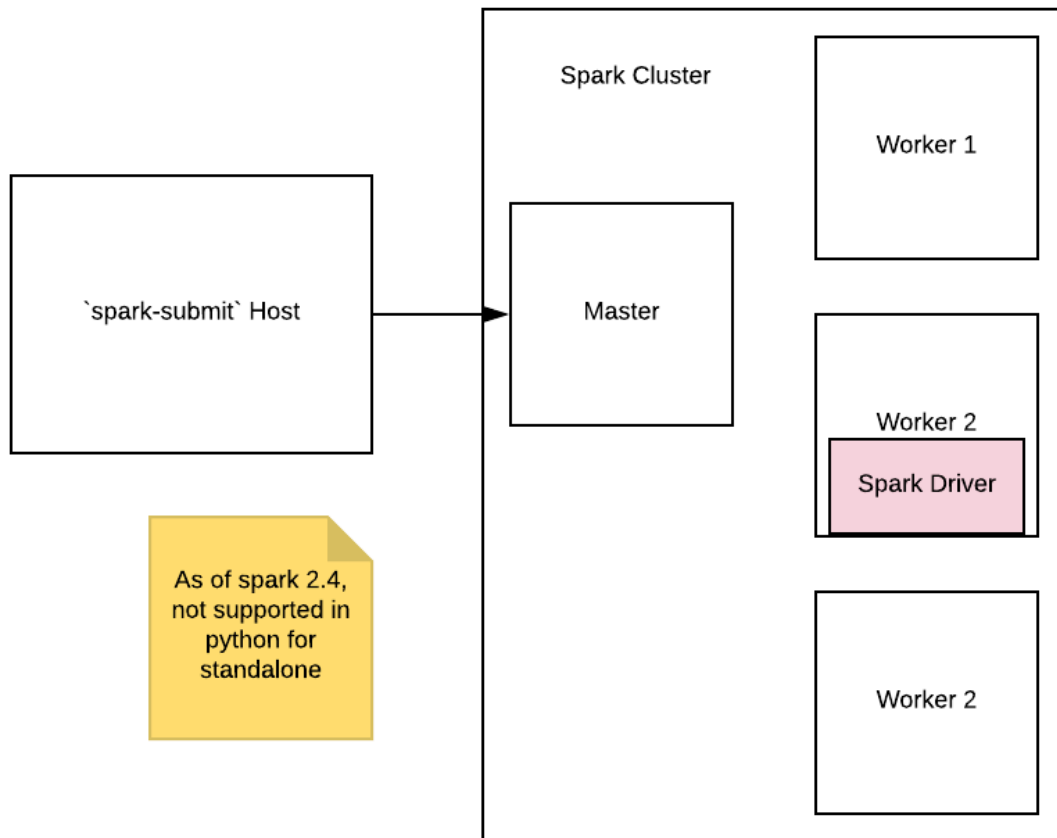
- Driver runs on the **machine where spark-submit is executed**
- Good for:
  - Development
  - Debugging
- Logs are visible in the terminal

---

## Cluster Mode

```
--deploy-mode cluster
```

--deploy-mode: cluster



- Driver runs on a **worker node**
- spark-submit exits after submission
- Used in:
  - Production
  - Automated pipelines

Note: Cluster mode is not supported in spark standalone cluster.

## Running PySpark Jobs

```
spark-submit \  
--master spark://spark-master:7077 \  
job.py
```

Inside `job.py`:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Demo").getOrCreate()
```

## Running JAR (Scala / Java) Jobs

```
spark-submit \  
--class com.demo.Main \  
--master spark://spark-master:7077 \  
app.jar
```

## Passing Arguments to Spark Job

```
spark-submit job.py input_path output_path
```

Inside Python:

```
import sys  
input_path = sys.argv[1]  
output_path = sys.argv[2]
```

## Resource Configuration

```
spark-submit \  
--executor-memory 4g \  
--executor-cores 2 \  
--num-executors 4 \  
job.py
```

What this means:

- 4 executors
- Each executor has 2 cores
- Each executor has 4 GB memory

## Setting Spark Configs

```
spark-submit \  
--conf spark.sql.shuffle.partitions=200 \  
--conf spark.executor.memoryOverhead=1g \  
job.py
```

## Spark configuration order of preference (highest → lowest)

### 1 `spark-submit` command line options (highest priority)

Example:

```
spark-submit \  
--executor-memory 4g \  
--executor-cores 2 \  
--conf spark.sql.shuffle.partitions=10 \  
app.py
```

Anything passed here **overrides everything else**.

### 2 `SparkSession.builder.config()` in code

Example:

```
SparkSession.builder \  
  .config("spark.executor.memory", "2g") \  
  .config("spark.sql.shuffle.partitions", "5")
```

Used **only if not overridden by spark-submit**.

### 3 `spark-defaults.conf`

Located in:

```
$SPARK_HOME/conf/spark-defaults.conf
```

Applies to:

- all applications
- unless overridden by 1 or 2

#### 4 Spark internal defaults (lowest priority)

Example:

- `spark.sql.shuffle.partitions = 200`
- `spark.default.parallelism`

Used only if nothing else is set.

## Packaging Dependencies

### Python Libraries

```
spark-submit \  
--py-files utils.zip \  
job.py
```

### External JARs

```
spark-submit \  
--jars mysql-connector.jar \  
job.py
```

## spark-submit vs Notebook Execution

| Notebook          | spark-submit        |
|-------------------|---------------------|
| Interactive       | Batch               |
| Driver = notebook | Driver = new JVM    |
| Not production    | Production standard |
| For learning      | For real jobs       |

## Running spark-submit from Jupyter

You **can** run:

```
!spark-submit --master spark://spark-master:7077 job.py
```

Important:

- Jupyter is **not** the driver
- spark-submit starts a **new Spark application**

---

## Interview One Liner

spark-submit launches a new Spark application by starting a driver process, requesting executors from the cluster manager, and executing the job in either client or cluster mode.

---

## One Line Summary

If Spark is a car, spark-submit is the ignition key.

```
# notebook
!spark-submit \
--master spark://spark-master:7077 \
--deploy-mode client \
job.py

# terminal
docker exec -it spark-master bash
# create job.py in master container
/opt/spark/bin/spark-submit --master spark://spark-master:7077 --deploy-mode client job.py
```